DOCUMENTACIÓN TÉCNICA: API REST CON NEXT.JS, PRISMA Y SQL SERVER

Proyecto: Sistema de Autenticación y Gestión de Usuarios
Tecnologías: Next.js 15, Prisma ORM, SQL Server, TypeScript
Autor: Jhusseth Sanchez
Fecha: 3 de marzo de 2026

1. Introducción

Este documento describe la arquitectura e implementación de un API REST completo para autenticación y gestión de usuarios (CRUD) construido sobre Next.js App Router, utilizando Prisma ORM como capa de abstracción de base de datos y SQL Server como motor de persistencia.

1.1 Objetivos del Sistema

- Proveer endpoints seguros para registro, login y logout de usuarios
- Implementar CRUD completo de usuarios con autorización basada en roles
- Utilizar JSON Web Tokens (JWT) almacenados en cookies HttpOnly para gestión de sesiones
- Abstraer el acceso a datos mediante Prisma ORM para portabilidad entre motores SQL
- Aplicar mejores prácticas de seguridad según OWASP API Security Top 10

1.2 Arquitectura General

El sistema sigue una arquitectura de capas:

- **Capa de Presentación:** Cliente Next.js (frontend React)
- **Capa de API:** Route Handlers en App Router (app/api/*)
- **Capa de Lógica de Negocio:** Funciones de autenticación, validación (Zod), hashing (bcrypt)
- **Capa de Acceso a Datos:** Prisma Client
- **Capa de Persistencia:** SQL Server (MSSQL)

2. ¿Qué es Prisma ORM?

Prisma es un **Object-Relational Mapper (ORM)** de próxima generación para Node.js y TypeScript que proporciona una capa de abstracción type-safe entre tu aplicación y la base de datos[1].

2.1 Características Principales

- **Type Safety:** Genera tipos TypeScript automáticamente desde tu esquema de base de datos
- **Prisma Schema:** Lenguaje declarativo (DSL) para modelar tu estructura de datos
- **Prisma Client:** Cliente auto-generado con queries type-safe
- **Prisma Migrate:** Sistema de migraciones versionadas para evolución de esquema
- **Multi-Database Support:** Soporta PostgreSQL, MySQL, SQL Server, SQLite, MongoDB y CockroachDB

2.2 Ventajas sobre SQL Directo

Aspecto	SQL Directo	Prisma ORM
Type Safety	No (strings crudos)	Sí (tipos generados)
Migraciones	Manuales	Automáticas y versionadas
Portabilidad	Dependiente del motor	Agnóstico de DB
Developer Experience	Query builders manuales	IntelliSense completo
Relaciones	JOINs manuales	Navegación de objetos

Table 1: Comparativa Prisma vs SQL directo

2.3 Componentes del Ecosistema Prisma

Prisma Schema (`schema.prisma`):
Archivo declarativo que define modelos de datos, relaciones, providers de base de datos y generadores de cliente.

Prisma Client:
Cliente auto-generado que expone métodos CRUD type-safe basados en tu esquema. Se regenera con `npx prisma generate` tras cambios en el esquema.

Prisma Migrate:
Herramienta CLI que genera migraciones SQL desde cambios en el esquema y las aplica a la base de datos de forma versionada.

Prisma Studio:
GUI visual para explorar y manipular datos (accesible con `npx prisma studio`).

2.4 ¿Por Qué Prisma para Este API?

- **Seguridad:** Previene SQL injection mediante queries parametrizadas automáticamente
- **Productividad:** Reduce código boilerplate en ~40% comparado con query builders tradicionales[1]
- **Mantenibilidad:** Esquema centralizado facilita refactorización y auditoría
- **Portabilidad:** Cambiar de SQL Server a PostgreSQL requiere solo modificar el provider y connection string

3. Instalación de Dependencias

3.1 Dependencias de Producción

Ejecutar en la raíz del proyecto Next.js:

npm install @prisma/client @prisma/adapter-mssql bcryptjs zod jsonwebtoken cookie

Descripción de cada paquete:

- `@prisma/client`: Cliente Prisma auto-generado para queries type-safe
- `@prisma/adapter-mssql`: Adaptador específico para SQL Server requerido por Prisma 5+[2]
- `bcryptjs`: Librería de hashing para encriptación de contraseñas (implementación pura JS de bcrypt)
- `zod`: Librería de validación de esquemas con inferencia de tipos TypeScript
- `jsonwebtoken`: Implementación de JWT (RFC 7519) para generación y verificación de tokens
- `cookie`: Utilidad para parseo y serialización de cookies HTTP

3.2 Dependencias de Desarrollo

npm install -D prisma @types/mssql @types/bcryptjs @types/jsonwebtoken @types/cookie

Descripción:

- `prisma`: CLI de Prisma para migraciones, generación de cliente y studio
- `@types/mssql`: Definiciones TypeScript para el driver MSSQL
- `@types/bcryptjs`: Tipado para bcryptjs
- `@types/jsonwebtoken`: Tipado para jsonwebtoken
- `@types/cookie`: Tipado para cookie

3.3 Verificación de Instalación

npx prisma --version

Debe mostrar la versión instalada (ej: `prisma: 5.x.x`).

4. Configuración de SQL Server

4.1 Opción A: SQL Server con Docker (Desarrollo)

Levantar contenedor SQL Server 2019:

docker run -e "ACCEPT_EULA=Y"
-e "MSSQL_SA_PASSWORD=YourStrong!Passw0rd"
-p 1433:1433
--name mssql-dev
-d mcr.microsoft.com/mssql/server:2019-latest

Parámetros:

- `ACCEPT_EULA=Y`: Acepta términos de licencia
- `MSSQL_SA_PASSWORD`: Contraseña del usuario `sa` (admin)
- `-p 1433:1433`: Mapea puerto estándar de SQL Server
- `--name mssql-dev`: Nombre del contenedor

Verificar que el contenedor está corriendo:

docker ps | grep mssql-dev

4.2 Opción B: Azure SQL Database (Producción)

Si usas Azure SQL, obtén la connection string desde el portal Azure:

Server=tcp:myserver.database.windows.net,1433;Initial Catalog=mydb;Persist Security Info=False;User ID=myadmin;Password=mypassword;MultipleActiveResultSets=False;Encrypt=True;TrustServerCertificate=False;Connection Timeout=30;

4.3 Crear Base de Datos

Conéctate con Azure Data Studio o SQL Server Management Studio y ejecuta:

```
CREATE DATABASE nextjs_api_db;
GO
```

5. Inicialización de Prisma

5.1 Comando de Inicialización

Desde la raíz del proyecto:

npx prisma init

Resultado: Crea dos archivos:

- `prisma/schema.prisma`: Esquema de datos
- `.env`: Variables de entorno (si no existe)

5.2 Configuración de Variables de Entorno

Editar archivo .env en la raíz:

Connection string para SQL Server local (Docker)

```
DATABASE_URL="sqlserver://localhost:1433;database=nextjs_api_db;user
=sa;password=YourStrong!Passw0rd;trustServerCertificate=true;encrypt
=true"
```

Secret para firma de JWT (generar con openssl rand -base64 32)

```
JWT_SECRET="tu-clave-secreta-larga-y-aleatoria-de-al-menos-32-
caracteres"
```

Entorno Node.js

```
NODE_ENV="development"
```

Consideraciones de seguridad:

- Nunca commitear .env a git (agregarlo a .gitignore)
- En producción, usar variables de entorno del proveedor cloud (Vercel, Azure, AWS)
- trustServerCertificate=true solo en desarrollo; en producción usar certificados válidos

5.3 Definición del Esquema Prisma

Editar prisma/schema.prisma:

```
generator client {
provider = "prisma-client-js"
}

datasource db {
provider = "sqlserver"
url = env("DATABASE_URL")
}

model User {
id String @id @default(cuid())
email String @unique
password String
userName String?
firstName String?
lastName String?
role String @default("USER")
```

```
createdAt DateTime @default(now())
updatedAt DateTime @updatedAt

@@map("users")
}
```

Explicación de atributos:

- @id: Marca el campo como clave primaria
- @default(cuid()): Genera CUID (Collision-resistant Unique ID) automáticamente
- @unique: Crea índice único (constraint)
- String?: El símbolo ? indica campo opcional (nullable)
- @default("USER"): Valor por defecto
- @updatedAt: Prisma actualiza automáticamente este timestamp en cada modificación
- @@map("users"): Mapea el modelo a la tabla "users" en SQL Server

5.4 Crear Migración Inicial

```
npx prisma migrate dev --name init
```

Proceso ejecutado:

1. Genera archivo de migración SQL en prisma/migrations/
2. Aplica la migración a la base de datos (crea tabla users)
3. Genera/actualiza Prisma Client

Resultado esperado:

Your database is now in sync with your schema.
✓ Generated Prisma Client to ./node_modules/@prisma/client

5.5 Generar Cliente Prisma

Si solo modificas el schema sin migrar:

npx prisma generate

Esto regenera el cliente TypeScript con los nuevos tipos.

6. Implementación del Cliente Prisma

6.1 Patrón Singleton para Next.js

Crear archivo src/lib/prisma.ts:

```
import { PrismaClient } from '@prisma/client';

const globalForPrisma = globalThis as unknown as {
prisma?: PrismaClient
};

export const prisma =
globalForPrisma.prisma ??
new PrismaClient({
log: process.env.NODE_ENV === 'development'
? ['query', 'error', 'warn']
: ['error'],
});

if (process.env.NODE_ENV !== 'production') {
globalForPrisma.prisma = prisma;
}
```

Explicación:

En Next.js dev mode, el hot-reload puede instanciar múltiples `PrismaClient`, agotando conexiones. El patrón singleton previene esto reutilizando la instancia global[3].

Configuración de logs:

- Desarrollo: registra queries, errores y warnings
- Producción: solo errores (performance)

6.2 Uso del Cliente en Route Handlers

Importar en cualquier API route:

import { prisma } from '@/lib/prisma';

const users = await prisma.user.findMany();

7. Implementación de Autenticación

7.1 Módulo de Helpers de Autenticación

Crear `src/lib/auth.ts`:

```
import jwt from 'jsonwebtoken';
import { cookies } from 'next/headers';

const JWT_SECRET = process.env.JWT_SECRET!;
const ACCESS_TOKEN_EXPIRY = '15m';
const REFRESH_TOKEN_EXPIRY = '7d';

interface JwtPayload {
sub: string;
role: string;
}
```

```
export function signAccessToken(payload: JwtPayload): string {
return jwt.sign(payload, JWT_SECRET, {
expiresIn: ACCESS_TOKEN_EXPIRY,
issuer: 'nextjs-api',
audience: 'nextjs-app',
});
}

export function signRefreshToken(payload: { sub: string }): string {
return jwt.sign(payload, JWT_SECRET, {
expiresIn: REFRESH_TOKEN_EXPIRY,
issuer: 'nextjs-api',
audience: 'nextjs-app',
});
}

export function setAuthCookies(
accessToken: string,
refreshToken?: string
): void {
const store = cookies();

store.set('accessToken', accessToken, {
httpOnly: true,
secure: process.env.NODE_ENV === 'production',
sameSite: 'lax',
path: '/',
maxAge: 60 * 15, // 15 minutos
});

if (refreshToken) {
store.set('refreshToken', refreshToken, {
httpOnly: true,
secure: process.env.NODE_ENV === 'production',
sameSite: 'lax',
path: '/',
maxAge: 60 * 60 * 24 * 7, // 7 días
});
}
}

export function clearAuthCookies(): void {
const store = cookies();
store.set('accessToken', '', {
httpOnly: true,
secure: true,
sameSite: 'lax',
path: '/',
maxAge: 0
});
store.set('refreshToken', '', {
httpOnly: true,
secure: true,
sameSite: 'lax',
```

```
path: '/',
maxAge: 0
});
}

export function getUserFromRequestCookie(): JwtPayload | null {
const token = cookies().get('accessToken')?.value;
if (!token) return null;

try {
const decoded = jwt.verify(token, JWT_SECRET, {
issuer: 'nextjs-api',
audience: 'nextjs-app',
}) as jwt.JwtPayload;

return {
  sub: decoded.sub as string,
  role: decoded.role as string
};


} catch (error) {
return null;
}
}
```

Consideraciones de seguridad:

- **HttpOnly:** Previene acceso a cookies desde JavaScript (mitiga XSS)[4]
- **Secure:** Solo transmite cookies por HTTPS en producción[4]
- **SameSite=lax:** Protección contra CSRF básica[4]
- **Expiry corto para Access Token:** Limita ventana de exposición si el token es comprometido
- **Refresh Token:** Permite renovación sin re-login durante 7 días

8. Route Handlers: Autenticación

8.1 POST /api/auth/register

Crear src/app/api/auth/register/route.ts:

```
import { NextResponse } from 'next/server';
import { z } from 'zod';
import bcrypt from 'bcryptjs';
import { prisma } from '@/lib/prisma';

const RegisterSchema = z.object({
email: z.string().email('Email inválido'),
password: z.string()
.min(8, 'Contraseña debe tener mínimo 8 caracteres')
.regex(/[A-Z]/, 'Debe contener al menos una mayúscula')
```

```
.regex(/[0-9]/, 'Debe contener al menos un número'),
name: z.string().min(1, 'Nombre requerido').optional(),
});

export async function POST(request: Request) {
try {
const body = await request.json();
const parsed = RegisterSchema.safeParse(body);

if (!parsed.success) {
  return NextResponse.json(
    {
      message: 'Validación fallida',
      issues: parsed.error.issues
    },
    { status: 400 }
  );
}

const { email, password, name } = parsed.data;

const existing = await prisma.user.findUnique({
  where: { email }
});

if (existing) {
  return NextResponse.json(
    { message: 'Email ya registrado' },
    { status: 409 }
  );
}

const hashedPassword = await bcrypt.hash(password, 12);

const user = await prisma.user.create({
  data: {
    email,
    password: hashedPassword,
    name
  },
  select: {
    id: true,
    email: true,
    name: true,
    role: true,
    createdAt: true
  },
});

return NextResponse.json(
  { user },
  { status: 201 }
);
```

```
} catch (error) {
console.error('Register error:', error);
return NextResponse.json(
{ message: 'Error interno del servidor' },
{ status: 500 }
);
}
}
```

Puntos clave:

- **Validación con Zod:** Schema declarativo con mensajes de error personalizados
- **Bcrypt rounds=12:** Balance entre seguridad y performance (OWASP recomienda 10-12)[5]
- **Select explícito:** Nunca retornar el campo `password` hasheado en respuestas
- **Código de estado 409:** Conflict para emails duplicados (semántica HTTP correcta)

8.2 POST /api/auth/login

Crear `src/app/api/auth/login/route.ts`:

```
import { NextResponse } from 'next/server';
import { z } from 'zod';
import bcrypt from 'bcryptjs';
import { prisma } from '@/lib/prisma';
import {
signAccessToken,
signRefreshToken,
setAuthCookies
} from '@/lib/auth';

const LoginSchema = z.object({
email: z.string().email(),
password: z.string().min(1),
});

export async function POST(request: Request) {
try {
const body = await request.json();
const parsed = LoginSchema.safeParse(body);

if (!parsed.success) {
  return NextResponse.json(
    { message: 'Credenciales inválidas' },
    { status: 401 }
  );
}

const { email, password } = parsed.data;

const user = await prisma.user.findUnique({
  where: { email }
});
```

```
if (!user) {
  return NextResponse.json(
    { message: 'Credenciales inválidas' },
    { status: 401 }
  );
}

const isPasswordValid = await bcrypt.compare(
  password,
  user.password
);

if (!isPasswordValid) {
  return NextResponse.json(
    { message: 'Credenciales inválidas' },
    { status: 401 }
  );
}

const accessToken = signAccessToken({
  sub: user.id,
  role: user.role
});

const refreshToken = signRefreshToken({
  sub: user.id
});

setAuthCookies(accessToken, refreshToken);

return NextResponse.json({
  user: {
    id: user.id,
    email: user.email,
    name: user.name,
    role: user.role
  },
});


} catch (error) {
console.error('Login error:', error);
return NextResponse.json(
{ message: 'Error interno del servidor' },
{ status: 500 }
);
}
}
```

Estrategia de mensajes de error:

Usar mensaje genérico "Credenciales inválidas" tanto para email no existente como contraseña incorrecta. Esto previene enumeración de usuarios (OWASP A07:2021 - Identification and Authentication Failures)[4].

8.3 POST /api/auth/logout

Crear src/app/api/auth/logout/route.ts:

```
import { NextResponse } from 'next/server';
import { clearAuthCookies } from '@/lib/auth';

export async function POST() {
clearAuthCookies();
return NextResponse.json({
message: 'Logout exitoso'
});
}
```

9. Route Handlers: CRUD de Usuarios

9.1 GET/POST /api/users

Crear src/app/api/users/route.ts:

```
import { NextResponse } from 'next/server';
import { z } from 'zod';
import bcrypt from 'bcryptjs';
import { prisma } from '@/lib/prisma';
import { getUserFromRequestCookie } from '@/lib/auth';

const CreateUserSchema = z.object({
email: z.string().email(),
password: z.string().min(8),
name: z.string().min(1).optional(),
role: z.enum(['USER', 'ADMIN']).optional(),
});

export async function GET(request: Request) {
const session = getUserFromRequestCookie();

if (!session) {
return NextResponse.json(
{ message: 'No autenticado' },
{ status: 401 }
);
}

if (session.role !== 'ADMIN') {
return NextResponse.json(
{ message: 'Acceso denegado' },
{ status: 403 }
```

```
);
}

const { searchParams } = new URL(request.url);
const page = parseInt(searchParams.get('page') ?? '1');
const limit = parseInt(searchParams.get('limit') ?? '10');
const skip = (page - 1) * limit;

const [users, total] = await Promise.all([
prisma.user.findMany({
select: {
id: true,
email: true,
name: true,
role: true,
createdAt: true
},
orderBy: { createdAt: 'desc' },
skip,
take: limit,
}),
prisma.user.count(),
]);

return NextResponse.json({
users,
pagination: {
page,
limit,
total,
totalPages: Math.ceil(total / limit),
},
});
}

export async function POST(request: Request) {
const session = getUserFromRequestCookie();

if (!session) {
return NextResponse.json(
{ message: 'No autenticado' },
{ status: 401 }
);
}

if (session.role !== 'ADMIN') {
return NextResponse.json(
{ message: 'Acceso denegado' },
{ status: 403 }
);
}

try {
const body = await request.json();
const parsed = CreateUserSchema.safeParse(body);
```

```
if (!parsed.success) {
  return NextResponse.json(
    {
      message: 'Validación fallida',
      issues: parsed.error.issues
    },
    { status: 400 }
  );
}

const { email, password, name, role } = parsed.data;

const existing = await prisma.user.findUnique({
  where: { email }
});

if (existing) {
  return NextResponse.json(
    { message: 'Email ya registrado' },
    { status: 409 }
  );
}

const hashedPassword = await bcrypt.hash(password, 12);

const user = await prisma.user.create({
  data: {
    email,
    password: hashedPassword,
    name,
    role: role ?? 'USER'
  },
  select: {
    id: true,
    email: true,
    name: true,
    role: true,
    createdAt: true
  },
});

return NextResponse.json(
  { user },
  { status: 201 }
);


} catch (error) {
console.error('Create user error:', error);
return NextResponse.json(
{ message: 'Error interno del servidor' },
{ status: 500 }
);
```

```
}
}
```

Implementación de paginación:

La paginación reduce carga en BD y mejora performance. SQL Server con Prisma usa OFFSET/FETCH internamente[3].

9.2 GET/PATCH/DELETE /api/users/[id]

Crear src/app/api/users/[id]/route.ts:

```
import { NextResponse } from 'next/server';
import { z } from 'zod';
import bcrypt from 'bcryptjs';
import { prisma } from '@/lib/prisma';
import { getUserFromRequestCookie } from '@/lib/auth';

const UpdateUserSchema = z.object({
email: z.string().email().optional(),
name: z.string().min(1).optional(),
role: z.enum(['USER', 'ADMIN']).optional(),
password: z.string().min(8).optional(),
});

interface RouteContext {
params: Promise<{ id: string }>;
}

export async function GET(
request: Request,
context: RouteContext
) {
const session = getUserFromRequestCookie();

if (!session) {
return NextResponse.json(
{ message: 'No autenticado' },
{ status: 401 }
);
}

const { id } = await context.params;

const canRead = session.role === 'ADMIN' || session.sub === id;

if (!canRead) {
return NextResponse.json(
{ message: 'Acceso denegado' },
{ status: 403 }
);
}

const user = await prisma.user.findUnique({
where: { id },
```

```
select: {
id: true,
email: true,
name: true,
role: true,
createdAt: true,
updatedAt: true,
},
});

if (!user) {
return NextResponse.json(
{ message: 'Usuario no encontrado' },
{ status: 404 }
);
}

return NextResponse.json({ user });
}

export async function PATCH(
request: Request,
context: RouteContext
) {
const session = getUserFromRequestCookie();

if (!session) {
return NextResponse.json(
{ message: 'No autenticado' },
{ status: 401 }
);
}

const { id } = await context.params;

try {
const body = await request.json();
const parsed = UpdateUserSchema.safeParse(body);

if (!parsed.success) {
  return NextResponse.json(
    {
      message: 'Validación fallida',
      issues: parsed.error.issues
    },
    { status: 400 }
  );
}

const isAdmin = session.role === 'ADMIN';
const isSelf = session.sub === id;

if (!isAdmin && !isSelf) {
  return NextResponse.json(
    { message: 'Acceso denegado' },
```

```
    { status: 403 }
  );
}

const data: any = { ...parsed.data };

// Solo admin puede cambiar roles
if (!isAdmin && data.role) {
  delete data.role;
}

// Hash password si se está actualizando
if (data.password) {
  data.password = await bcrypt.hash(data.password, 12);
}

const user = await prisma.user.update({
  where: { id },
  data,
  select: {
    id: true,
    email: true,
    name: true,
    role: true,
    updatedAt: true
  },
});

return NextResponse.json({ user });


} catch (error: any) {
if (error.code === 'P2025') {
return NextResponse.json(
{ message: 'Usuario no encontrado' },
{ status: 404 }
);
}

console.error('Update user error:', error);
return NextResponse.json(
  { message: 'Error interno del servidor' },
  { status: 500 }
);


}
}

export async function DELETE(
request: Request,
context: RouteContext
) {
const session = getUserFromRequestCookie();
```

```
if (!session) {
return NextResponse.json(
{ message: 'No autenticado' },
{ status: 401 }
);
}

if (session.role !== 'ADMIN') {
return NextResponse.json(
{ message: 'Acceso denegado' },
{ status: 403 }
);
}

const { id } = await context.params;

try {
await prisma.user.delete({ where: { id } });

return NextResponse.json({
  message: 'Usuario eliminado exitosamente'
});


} catch (error: any) {
if (error.code === 'P2025') {
return NextResponse.json(
{ message: 'Usuario no encontrado' },
{ status: 404 }
);
}

console.error('Delete user error:', error);
return NextResponse.json(
  { message: 'Error interno del servidor' },
  { status: 500 }
);


}
}
```

Control de acceso granular:

La autorización verifica dos condiciones: (1) usuarios pueden ver/editar su propio perfil, (2) admins pueden ver/editar/eliminar cualquier usuario (OWASP A01:2021 - Broken Object Level Authorization)[4].

10. Estructura Final del Proyecto

```
nextjs-app/
├── src/
```

```
│  ├── app/
│  │   ├── api/
│  │   │   ├── auth/
│  │   │   │   ├── login/
│  │   │   │   │   └── route.ts
│  │   │   │   ├── register/
│  │   │   │   │   └── route.ts
│  │   │   │   ├── logout/
│  │   │   │   └── route.ts
│  │   │   ├── users/
│  │   │   ├── route.ts
│  │   │   ├── [id]/
│  │   │   └── route.ts
│  │   └── ...
│  ├── lib/
│  ├── prisma.ts
│  └── auth.ts
├── prisma/
│  ├── schema.prisma
│  ├── migrations/
│  ├── 20260301_init/
│  └── migration.sql
├── .env
├── .env.example
├── package.json
└── tsconfig.json
```

11. Testing de Endpoints

11.1 Registro de Usuario

```
curl -X POST http://localhost:3000/api/auth/register
-H "Content-Type: application/json"
-d '{
"email": "admin@test.com",
"password": "Admin123",
"name": "Administrator"
}'

Respuesta esperada (201):

{
"user": {
"id": "clx1a2b3c4d5e6f7g8h9i0j1",
"email": "admin@test.com",
"name": "Administrator",
"role": "USER",
"createdAt": "2026-03-01T20:00:00.000Z"
}
}
```

11.2 Login

```
curl -i -X POST http://localhost:3000/api/auth/login
-H "Content-Type: application/json"
-d '{
"email": "admin@test.com",
"password": "Admin123"
}'
```

```
Respuesta esperada (200 + cookies):
```

```
HTTP/1.1 200 OK
Set-Cookie: accessToken=eyJhbGc...; HttpOnly; Secure; SameSite=Lax
Set-Cookie: refreshToken=eyJhbGc...; HttpOnly; Secure; SameSite=Lax
Content-Type: application/json

{
"user": {
"id": "clx1a2b3c4d5e6f7g8h9i0j1",
"email": "admin@test.com",
"name": "Administrator",
"role": "USER"
}
}
```

11.3 Listar Usuarios (Requiere ADMIN)

```
curl -X GET http://localhost:3000/api/users
-H "Cookie: accessToken=eyJhbGc..."
```

11.4 Actualizar Usuario

```
curl -X PATCH
http://localhost:3000/api/users/clx1a2b3c4d5e6f7g8h9i0j1
-H "Content-Type: application/json"
-H "Cookie: accessToken=eyJhbGc..."
-d '{
"name": "Admin Updated"
}'
```

11.5 Eliminar Usuario

```
curl -X DELETE
http://localhost:3000/api/users/clx1a2b3c4d5e6f7g8h9i0j1
-H "Cookie: accessToken=eyJhbGc..."
```

12. Seguridad y Mejores Prácticas

12.1 Mitigación de Riesgos OWASP API Top 10

Riesgo	Implementación	Código
A01 - BOLA	Validar ownership en cada endpoint	getUserFromRequestCookie
A02 - Auth rota	JWT + HttpOnly cookies	auth.ts
A03 - Exceso de datos	Select explícito en Prisma	Nunca retornar password
A04 - Rate limiting	Pendiente	middleware.ts futuro
A05 - BFLA	Validar roles (ADMIN/USER)	if (role !== 'ADMIN')
A07 - Injection	Queries parametrizadas Prisma	prisma.user.findUnique
A08 - Logs sensibles	No loggear passwords/tokens	console.error genérico

Table 2: Controles de seguridad implementados

12.2 Recomendaciones Adicionales

1. **Rate Limiting:** Implementar con `@upstash/ratelimit` para prevenir brute-force
2. **Refresh Token Rotation:** Regenerar refresh token en cada uso
3. **Auditoría:** Loggear acciones críticas (login, cambio de rol, eliminación)
4. **Validación de Email:** Implementar confirmación por correo electrónico
5. **2FA:** Considerar autenticación de dos factores para usuarios ADMIN
6. **CORS:** Configurar políticas restrictivas en producción
7. **CSP:** Content Security Policy headers
8. **Helmet:** Headers de seguridad adicionales

13. Migración a Otras Bases de Datos

13.1 PostgreSQL

Cambiar en `schema.prisma`:

```
datasource db {
provider = "postgresql"
url = env("DATABASE_URL")
}
```

Connection string en `.env`:

```
DATABASE_URL="postgresql://user:password@localhost:5432/mydb?schema=
public"
```

Ejecutar migración:

```
npx prisma migrate dev
```

13.2 MySQL

```
datasource db {
provider = "mysql"
url = env("DATABASE_URL")
}
```

Connection string:

```
DATABASE_URL="mysql://user:password@localhost:3306/mydb"
```

Nota: El código de la aplicación no requiere cambios. Prisma abstrae las diferencias entre motores SQL.

14. Comandos Útiles de Prisma

Comando	Descripción
`npx prisma init`	Inicializa Prisma en el proyecto
`npx prisma generate`	Regenera Prisma Client tras cambios en schema
`npx prisma migrate dev`	Crea y aplica migración en desarrollo
`npx prisma migrate deploy`	Aplica migraciones pendientes en producción
`npx prisma studio`	Abre GUI web para explorar datos
`npx prisma db push`	Sincroniza schema sin crear migración (prototyping)
`npx prisma db seed`	Ejecuta seed script para datos iniciales
`npx prisma format`	Formatea schema.prisma
`npx prisma validate`	Valida sintaxis del schema

Table 3: Comandos CLI de Prisma

15. Troubleshooting Común

15.1 Error: "Can't reach database server"

Causa: SQL Server no está corriendo o connection string incorrecta.

Solución:

- Verificar que Docker container está corriendo: `docker ps`
- Testear conexión con Azure Data Studio

- Validar que puerto 1433 no está bloqueado por firewall
- Verificar credenciales en DATABASE_URL

15.2 Error: "PrismaClient unable to run in the browser"

Causa: Intentando usar Prisma Client en componente cliente (con `'use client'`).

Solución: Prisma solo funciona en server-side. Usar Route Handlers o Server Actions.

15.3 Error: "Type instantiation is excessively deep"

Causa: TypeScript sobrecargado con tipos complejos de Prisma.

Solución: Actualizar a TypeScript 5.1+ y reiniciar TS server en VSCode.

15.4 Migraciones Fallidas

Síntoma: Migration no se aplica y deja DB en estado inconsistente.

Solución:

npx prisma migrate resolve --rolled-back "20260301_migration_name"
npx prisma migrate dev

16. Deployment en Producción

16.1 Variables de Entorno en Vercel

En dashboard de Vercel, configurar:

```
DATABASE_URL="sqlserver://production-host:1433;database=prod_db;..."
JWT_SECRET="..." (generar con openssl rand -base64 64)
NODE_ENV="production"
```

16.2 Build y Deploy

```
npm run build
npx prisma generate
npx prisma migrate deploy

Vercel ejecuta automáticamente build commands. Agregar en
package.json:

{
"scripts": {
"postinstall": "prisma generate",
"build": "next build"
}
}
```

16.3 Connection Pooling

Para SQL Server en Azure, usar Prisma Data Proxy o configurar connection pool:

```
datasource db {
provider = "sqlserver"
url = env("DATABASE_URL")
relationMode = "prisma"
}

generator client {
provider = "prisma-client-js"
previewFeatures = ["driverAdapters"]
}
```

17. Conclusiones

Este sistema implementa un API REST completo siguiendo arquitectura de capas, aplicando principios SOLID y cumpliendo estándares de seguridad OWASP. Prisma ORM proporciona abstracción type-safe que reduce bugs en tiempo de ejecución y facilita mantenimiento evolutivo.

17.1 Ventajas Logradas

- **Type Safety:** TypeScript end-to-end elimina errores de tipado
- **Seguridad:** JWT HttpOnly + bcrypt + validación Zod + autorización granular
- **Portabilidad:** Cambio de DB sin modificar código de aplicación
- **Developer Experience:** IntelliSense, migraciones automáticas, Prisma Studio
- **Performance:** Connection pooling, queries optimizadas, paginación

17.2 Próximos Pasos Recomendados

1. Implementar refresh token rotation
2. Agregar rate limiting con Upstash Redis
3. Configurar logging estructurado (Winston/Pino)
4. Implementar testing (Jest + Supertest)
5. Documentar API con OpenAPI/Swagger
6. Configurar CI/CD con GitHub Actions
7. Implementar caché con Redis para queries frecuentes
8. Agregar monitoring (Sentry/DataDog)

Referencias

[1] Prisma. (2026). Prisma ORM with SQL Server Quickstart. https://www.prisma.io/docs/prisma-orm/quickstart/sql-server

[2] NPM. (2025). @prisma/adapter-mssql package documentation. https://www.npmjs.com/package/@prisma/adapter-mssql

[3] Next.js. (2026). Route Handlers - App Router Documentation. https://nextjs.org/docs/app/getting-started/route-handlers

[4] OWASP. (2023). API Security Top 10 - 2023. https://owasp.org/API-Security/editions/2023/en/0x11-t10/

[5] OWASP. (2023). Password Storage Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Apéndice A: Schema Completo con Relaciones

Ejemplo extendido con posts y comentarios:

```
model User {
  id        String   @id @default(cuid())
  firstName String
  lastName  String
  email     String   @unique
  userName  String   @unique
  password  String
  isActive  Boolean  @default(false)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  roleId    Int
  role      Role     @relation(fields: [roleId], references: [id])
}

model Role {
  id          Int          @id @default(autoincrement())
  name        String       @unique
  description String?
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt
  users       User[]
  permissions Permission[]
}

model Permission {
  id          Int      @id @default(autoincrement())
  key         String   @unique
  description String?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  roles       Role[]
}
```

Apéndice B: Seed Script

Crear prisma/seed.ts:

```
import { prisma } from '@/lib/prisma';
import bcrypt from 'bcryptjs';

async function main() {
  const permissions = [
    {key: 'dashboard:read' , description: 'Read Dashboard'} ,
    {key: 'users:read' , description: 'Read users'} ,
    {key: 'users:create' , description: 'Create users'} ,
    {key: 'users:update' , description: 'Update users'} ,
    {key: 'users:delete' , description: 'Delete users'} ,
  ];

  await prisma.permission.createMany({
    data: permissions
  });

  const admin = await prisma.role.upsert({
    where: {name: 'admin'} ,
    update: {} ,
    create: {name: 'admin' , description: 'Administrator'} ,
  });

  const user = await prisma.role.upsert({
    where: {name: 'user'} ,
    update: {} ,
    create: {name: 'user' , description: 'Default user'} ,
  });

  const viewer = await prisma.role.upsert({
    where: {name: 'viewer'} ,
    update: {} ,
    create: {name: 'viewer' , description: 'Viewer'} ,
  });

  const allPerms = await prisma.permission.findMany({
    where: {key: {in: permissions.map(p => p.key)}} ,
    select: {id: true} ,
  });

  await prisma.role.update({
    where: {id: admin.id} ,
    data: {
      permissions: {
        connect: allPerms ,
      } ,
    } ,
  });

  const readDashboard = await prisma.permission.findUnique({
```

```
    where: {key: 'dashboard:read'} ,
    select: {id: true} ,
  });

  await prisma.role.update({
    where: {id: user.id} ,
    data: {
      permissions: {
        set: readDashboard ? [readDashboard] : [] ,
      } ,
    } ,
  });

  const readPerms = await prisma.permission.findMany({
    where: {key: {in: ['dashboard:read' , 'users:read']}} ,
    select: {id: true} ,
  });

  await prisma.role.update({
    where: {id: viewer.id} ,
    data: {
      permissions: {
        connect: readPerms ,
      } ,
    } ,
  });

  const passwordPlain = process.env.SEED_TEST_PASSWORD ??
'admin123';
  const hashed = await bcrypt.hash(passwordPlain, 12);

  // Admin
  await prisma.user.upsert({
    where: { userName: 'admin' },
    update: {
      firstName: 'Admin',
      lastName: 'Test',
      email: 'admin@test.com',
      password: hashed,
      isActive: true,
      role: { connect: { id: admin.id } },
    },
    create: {
      firstName: 'Admin',
      lastName: 'Test',
      email: 'admin@test.com',
      userName: 'admin',
      password: hashed,
      isActive: true,
      role: { connect: { id: admin.id } },
    },
  });

  // Viewer
```

```
  await prisma.user.upsert({
    where: { userName: 'viewer' },
    update: {
      firstName: 'Viewer',
      lastName: 'Test',
      email: 'viewer@test.com',
      password: hashed,
      isActive: true,
      role: { connect: { id: viewer.id } },
    },
    create: {
      firstName: 'Viewer',
      lastName: 'Test',
      email: 'viewer@test.com',
      userName: 'viewer',
      password: hashed,
      isActive: true,
      role: { connect: { id: viewer.id } },
    },
  });

  // User
  await prisma.user.upsert({
    where: { userName: 'user' },
    update: {
      firstName: 'User',
      lastName: 'Test',
      email: 'user@test.com',
      password: hashed,
      isActive: true,
      role: { connect: { id: user.id } },
    },
    create: {
      firstName: 'User',
      lastName: 'Test',
      email: 'user@test.com',
      userName: 'user',
      password: hashed,
      isActive: true,
      role: { connect: { id: user.id } },
    },
  });
}

main()
    .then(() => prisma.$disconnect())
    .catch(async (e) => {
      console.error(e);
      await prisma.$disconnect();
      process.exit(1);
    });
```

Ejecutar:

```
npx prisma db seed
```